Unified Morphing Architectures: Revised Draft

Nov 3, 2026 · @G

Abstract

Unified Morphing Architectures (UMA) replace the stored program with a learned synthesiser that emits short, bounded instruction sequences on demand from the machine's current state. Each sequence passes a checker, runs in a fast burst on a RISC-V core with reconfigurable logic, and is then discarded. A recorded trace log of what was emitted, with real-time stamps, makes every run auditable and replayable.

UMA is best understood as an interpreter or JIT whose front end is a learned model instead of a bytecode stream. It does not aim to beat conventional execution on throughput. It proposes a different computational medium, in which the boundary between hardware and software is dissolved at the edges while proven paths harden into ordinary fast machinery. The same substrate can serve as a network switch, a phone or any other gadget, differing only in intent, peripherals and invariants, and units can be composed into coordinated hierarchies. A first simulation suggests viability depends mainly on the block-cache hit rate and the Checker's cost per block.

1. Introduction

Since the Von Neumann architecture, computing has run on a script-and-reader model. Software is built, frozen into a binary, loaded into memory, and fetched and executed in order. The costs are familiar: storage and memory-wall latency, exposure to static-analysis attacks, and algorithms bent to fit fixed silicon.

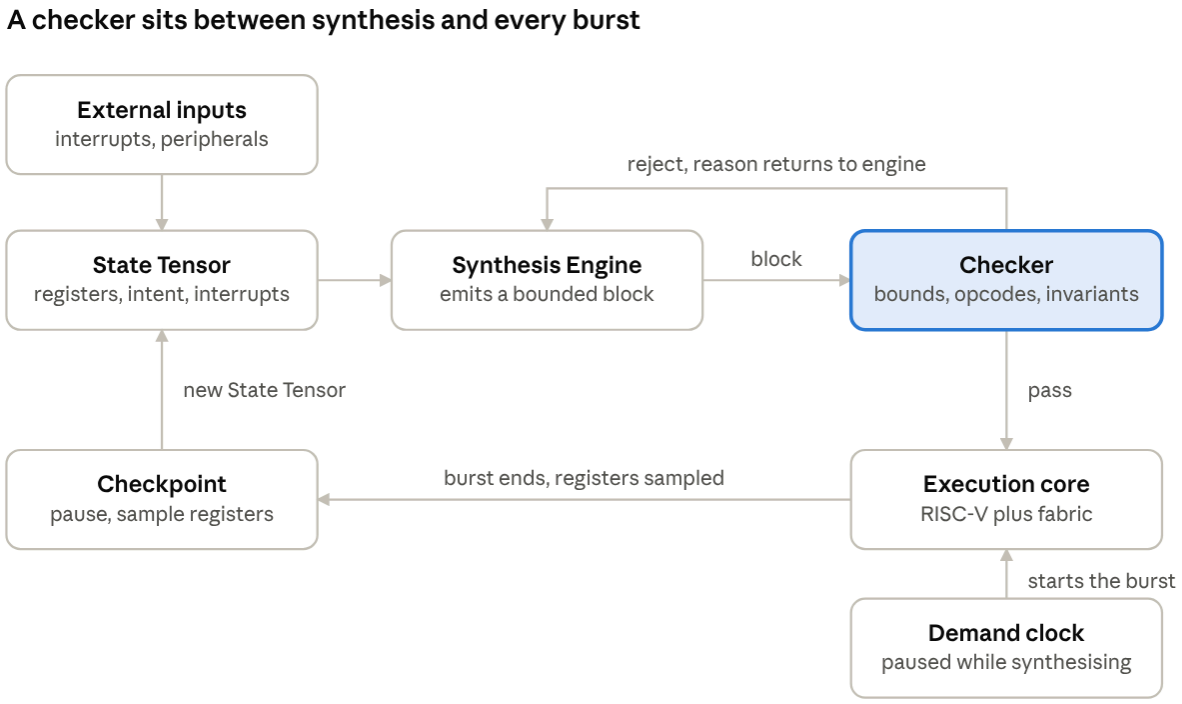
UMA asks what changes if instructions are generated at the moment they are needed, from the live state of the machine, rather than read from a stored program. Instruction streams become transient. They are produced, checked, executed and, if wanted, logged, but they are not authored in advance and shipped as files.

What this paper claims. The hardware/software boundary can be dissolved at the point where instructions are produced, and the resulting system is observable and reproducible because the emitted stream can be tapped, logged and replayed. The design starts from a minimum viable system and improves in stages (section 8).

What it does not claim. That information persisting in the machine disappears. Whatever shapes the synthesiser's behaviour (parameters, fabric configuration, analog state) is the program's content in another medium. UMA changes where that content lives and when instructions are formed, not whether the machine has behaviour that was specified somewhere. It also does not claim a throughput advantage over compiled code.

2. Architecture

Four components replace the conventional fetch-decode-execute control loop.



architecture loop · 6 components, 1 reject loop

A block must pass the Checker before the demand clock starts a burst, and the checkpoint feeds the next State Tensor.

State Tensor. In place of a program counter pointing into memory, execution state is a vector holding the architectural registers, a representation of current intent, and the live interrupt lines. It is the synthesiser's only input. Within a burst a small sequential index steps through the emitted instructions, so a counter still exists, but it indexes a block, not a program.

Synthesis Engine. A dedicated accelerator runs a model over a constrained token space of machine opcodes and operands. It predicts the next short block that moves the machine toward the current intent. How the model is realised (fixed weights in silicon, adaptive, analog, or embodied in the fabric) is left open by the architecture. UMA only requires that it consume a State Tensor and emit a bounded block. Where inference

latency matters, lightweight realisations such as highly quantised or binary neural networks, or small table-driven models, are natural candidates, but they are examples, not requirements.

Checker. Between synthesis and execution sits a verifier that accepts or rejects each block before the clock fires. It enforces memory bounds, permitted opcodes and fabric configurations, and task-specific invariants. This is the choke point that makes small, bounded generation safe and not merely small. Because a block has bounded length and only bounded loops with a stated trip count, it can be verified as deterministic by a mathematical checker, akin to an SMT solver or proof checker and possibly AI-assisted, which confirms that the block's effect on the State Tensor matches the intended transition before it runs. A rejected block returns to the engine with the reason as additional input. The Checker's cost per block sets a floor on the whole machine's speed (section 7), so its rule set must stay small and cheap to evaluate: bounds, permitted opcodes, structural resource limits, an execution-depth bound, fabric configurations and the intended-transition check.

Demand Clock. Execution is not driven by a fixed oscillator. The core rests in a paused, low-power state while the engine synthesises, and a hardware flag raises the clock edge when a checked block is ready. The machine therefore advances at the pace of synthesis plus checking, and stalls during that time by design, not by accident.

System	Clock	Program state	Execution style
Single-core	One fixed heartbeat	One program counter	Linear, one step at a time
Multi-core	One master clock, independent throttles	Several program counters	Parallel, simultaneous steps
UMA	Demand-driven asynchronous pulses	State Tensor plus in-burst index	Checked micro-bursts

3. Micro-Burst Execution

The engine never predicts a whole application. It emits a finite block. This paper starts from 4 to 16 instructions, but nothing in the architecture fixes the size (see section 8), and the machine cycles through four phases. Block size is a tradeoff: larger blocks amortise synthesis when nothing is cached, but smaller blocks hit the cache more often (section 7).

1. **Pulse.** The engine reads the State Tensor and emits a block.
2. **Check.** The Checker validates the block against bounds, opcode and fabric rules, and invariants. A failure loops back to the Pulse phase.

3. **Burst.** The clock runs at full speed through the block with a simple sequential index.
4. **Checkpoint.** The clock pauses, registers are sampled into a fresh State Tensor, and any interrupt or external change is folded into the next Pulse.

Reproducibility. Generation may be stochastic, but execution is reproducible because the emitted block is a concrete instruction sequence that can be recorded. A trace log of blocks together with the logged external inputs (interrupt vectors, peripheral and network data) that shaped each State Tensor is enough to replay a run exactly. The same tap supports logging, auditing, positive and negative feedback for training, and fault analysis, as in record-and-replay debuggers and tracing JITs.

Dual-track trace log. The trace log has two tracks, like a film strip with a timecode down its edge. A spatial track records which blocks ran and what they touched, with no time in it, so identical blocks are identical records and stay cacheable. A temporal track records real-time stamps taken by a hardwired Trace Tap at each demand-clock edge, from a hardware-isolated time reference. The inputs that shaped each State Tensor are logged with their stamps. In this paper the block cache holds admitted blocks for reuse and the trace log records what ran, and the two are separate. Details and cautions are in Appendix B.

Consequence. A persisted trace log is a recording of what the machine did, which is itself a software artifact. UMA's position is that code need not exist as an authored file before execution. It can exist afterwards, as a record.

4. The Ephemeral ISA

The execution fabric combines an open RISC-V base with reconfigurable logic.

Baked-in primitives. A hardwired core of standard RISC-V instructions (ADD, SUB, AND, OR and the rest of the base set) handles fundamental arithmetic at full efficiency with no routing overhead.

Custom extensions. For a hot or awkward operation, such as an unaligned multi-stage shift-and-accumulate, the engine can select a custom instruction in the RISC-V extension space instead of emitting several base instructions. The engine does this by emitting a macro-token that indexes a pre-staged table of custom operations in the fabric. It does not compile a raw micro-bitstream on the fly, which would carry heavy runtime validation costs and the hazards described below.

Reconfiguration cost. Partial reconfiguration of real FPGA fabric takes far longer than a 4 to 16 instruction burst, so rewiring per burst is only a baseline for comparison and pays off only when the custom op is reused many times. Three techniques keep reconfiguration off the hot path, the first being the pre-staged table above:

- **Pre-staged variants.** Candidate custom ops are held in neighbouring fabric regions, and "minting" an instruction means selecting one, a mux change, not reprogramming gates.
- **Coarse-grained reconfiguration.** On coarse-grained reconfigurable arrays a context switch costs cycles, not a bitstream load.
- **Promotion by use.** A custom op that keeps recurring is made permanent. Dissolving back to neutral fabric applies only to rare ones. In the simulation promotion narrowly beat pre-staging, but that result is sensitive to the assumed cost of the multiplexer switch.

Checked configurations. Any fabric configuration the engine proposes is validated by the Checker before it is applied. A malformed bitstream can create contention and damage hardware, so configurations are drawn from a vetted, parameterised set and not generated freely.

5. Interrupts and Asynchronous Events

Because execution proceeds in bounded bursts, an interrupt does not need to preempt mid-instruction. It alters the State Tensor, the current burst finishes, and at the next checkpoint the engine reads the interrupt vector as a high-priority input and synthesises a handler block. The prior intent path stays resident in the State Tensor's saved registers until the engine routes back to it.

Latency. Without a fast path, interrupt latency is the remaining burst plus a synthesis and check pass. That is acceptable for many workloads and unacceptable for hard real-time ones. The architecture therefore needs a small fixed fast path for urgent events, with synthesis reserved for the rest.

Asynchronous fast path. For urgent events, a dedicated path handles immediate hardware overrides. When an urgent interrupt line fires, a bypass multiplexer uncouples the Demand Clock from the Synthesis Engine and routes execution to a hardwired, immutable recovery block or a verified, pre-cached handler sequence. In-flight synthesis is abortable because it has changed no architectural state, so it is simply discarded. A burst already executing runs to its end, so worst-case fast-path latency is the remainder of one burst plus the bypass time. The simulation supports this: with the cache off, aborting in-flight synthesis cut worst-case latency from 12 to 15 µs to 16 to 42 ns, at the price of 7 to 17% more total time in discarded work. The recovery blocks and fast-path handlers are fixed code, which makes them part of the residual policy described in section 9.

State saving. Keeping the old state in inactive registers is still a form of context save. UMA simplifies it by tying it to burst boundaries, not removing it.

6. Optimisation Trajectory

The costs identified above are engineering problems that known techniques erode, and each points the same way: hot paths harden, novelty stays synthesised.

Cost	Mitigation	Effect
Synthesis latency	Cache frequently emitted blocks, keyed on a quantised State Tensor; synthesise only on a miss	Hot path approaches conventional hardware speed if the working set fits and only fully verified blocks are admitted (section 7)
Synthesis latency	Speculatively generate the next block while the current one executes	Little gain unless synthesis plus check takes about one burst time or less (section 7)
Reconfiguration time	Pre-staged variants, coarse-grained arrays	Minting an op costs cycles, not a bitstream load
Recurring custom ops	Promote to permanent hardware by usage	Dissolving applies only to rare ops
Interrupt latency	Fixed fast path for urgent events; handlers as cacheable blocks	Real-time cases served without synthesis

Where this lands. A block cache with a learned front end. The ephemeral path is the cold path, handling novelty, adaptation and rare cases. Proven paths harden into ordinary machinery. This is not a retreat from the idea. It is the stable form of it, in which the dissolution of the hardware/software boundary is real at the edges and the core is fast and checkable. The simulation in section 7 ranks these paths: the cache dominates, the interrupt fast path comes next, promotion after that, and speculation matters least.

Further paths.

- **Loop caching.** Cache whole loops, not fixed windows, so a hot loop becomes one cached unit. This is simulated in section 8 (Loops inside blocks).
- **Cheaper checking on the hit path.** A cached block was fully checked at admission, so a hit can skip the full check and run only a cheap integrity test. Checking can also be pipelined with synthesis.
- **State quantisation.** How coarsely the State Tensor is quantised sets the cache key space, and so the hit rate. This is the main unknown behind the simulation's best-case result.

- **Block libraries in hierarchies.** A parent could supply pre-verified blocks to its children, saving their cold-start synthesis. This conflicts with the rule that units exchange intent and not instruction streams, so it would need signed, locally re-checked blocks and is left open.

7. Simulation Findings

Cache hit rate and checker cost per block decide whether UMA runs near ordinary hardware speed or orders of magnitude slower. A small simulator (`uma_sim.py`, stdlib Python) models the pulse, check, burst and checkpoint loop, with the synthesiser, checker and fabric as black boxes. The synthesiser is an oracle with injected errors. The checker shadow-executes each block against the intended transition and applies static safety rules. The workload is 24 loop kernels with Zipf popularity over 2000 tasks. All timings are placeholder assumptions, so the results show shape, not absolute performance.

Question	Result
What does the block cache buy?	Slowdown against a conventional core falls from 226 to 788 times with no cache to about 3 times with 1024 entries, at 98 to 100% hit rate
Does burst size matter once cached?	Larger bursts cost more, because the hit rate falls. Out to 256 instructions the hit rate drops from 100% to 74% and slowdown rises from 3 to 30 times, with the best result near 16 (see section 8)
Does speculation help?	Only when synthesis plus check takes about one burst time (18 ns) or less. At 4,120 ns it changes nothing; at 16 ns it hides the pulse (2.17 times to 1.18 times)
What does the checker buy?	Uncached, full coverage gave zero wrong blocks and zero safety incidents up to 5% synthesiser error. At 90% coverage 58 wrong blocks got through; with safety rules only, 551
What does a cache do to checker escapes?	It cuts the count but makes survivors persistent: at 20% error and 50% coverage, 589 of 596 wrong blocks were replays of cached ones

Question	Result
Fabric strategy for a custom op?	Rewiring per burst took 729 ms against 1.76 ms with no custom op. Promotion was fastest (1.61 ms); pre-staging narrowly lost (1.80 ms) because its mux switch costs more than the 3 ns per element it saves
Interrupt latency?	Dominated by synthesis already in flight, not the remaining burst. With the cache off, a fast path that aborts in-flight synthesis cut the worst case from 12 to 15 µs to 16 to 42 ns, bounded by the remaining burst. With a cached handler, p50 was 0.012 µs at burst 4 against 4.4 µs uncached

Implications. The block working set must fit the cache, or the architecture is not viable on these assumptions. Because a cache turns an undetected error into a repeated one, only blocks that passed the full check should enter it. A fast path for urgent interrupts must be able to abort synthesis. The simulator now models this: it removes the synthesis wait but not the burst wait, and costs 7 to 17% more total time in discarded work compared with a fast path that waits for the checkpoint.

Limits. The synthesiser is an oracle, not a learned model. The checker compares against a known-good block, so the simulator says nothing about how hard it is to write the specification a real checker needs. Cached blocks are assumed reusable across data. Interrupt percentiles for cached runs rest on about 100 samples. The abort model assumes synthesis touches no architectural state, applies only to pulses and not to cache lookups, and cannot abort a running burst. The simulated cache is keyed on kernel identity and position, which is perfect knowledge. A real cache keyed on a quantised State Tensor will hit less often, so the roughly 3 times slowdown is a best case.

8. Minimum Viable System and Optimisation Paths

The core of UMA is deliberately small, and everything else is an optimisation area to pursue once the minimum system runs. This section sets out that minimum, the order in which improvements might follow, and how different devices would settle at different points.

Minimum viable system. Short straight-line blocks (4 to 16 instructions as a starting point), a Checker that enforces bounds and permitted opcodes and tests each block against the intended transition, a fixed interrupt fast path, and a trace log of blocks and

inputs. It is slow: uncached, the simulation puts it hundreds of times behind a conventional core. That is acceptable, because its job is to run correctly and to produce the trace log that every later step is built from.

Bootstrapping. The trace log is the instrument. Every emitted block and input is logged, so the running system exposes its own working set, hit rates and Checker costs, which are the numbers that decide the next step. Improvements are adopted in order of evidence, each only after the previous stage works.

Power and persistence. A UMA exists only while power is applied. It is instantiated at power-on from the hardwired intent and the Checker's invariants, its block cache starts empty, and everything is lost at power-off. The base system has no memory across power cycles and none is intended, so every boot is cold and pays its warm-up cost again. Anything persisted, such as a trace log written to a file, is the output of a higher-level function and not part of the architecture.

Hibernation as a line of enquiry. Future hardware may allow the whole machine to be held in a dormant state, as a bistable display holds an image with no power, and then resumed, progressed and frozen again. That would be a property of the underlying hardware, defined later, and not a requirement of UMA. Three things make it worth pursuing. Burst boundaries are natural freeze points, because the machine state is sampled into a fresh State Tensor at every checkpoint. A dormant machine is a retained machine and not a stored program, so the position above is unchanged. And it raises questions the base design does not: what exactly is retained (the State Tensor and memory only, or also the block cache and any adaptive engine state), how the Checker verifies the integrity of a dormant state on resume, how stale inputs are refreshed after a long gap, and how the trace log shows the gap, which the real-time stamps would record explicitly.

Stage	Adds	What the Checker must do	Simulated evidence
0	Minimum viable system	Bounds, opcodes, test against the intended transition	Uncached, 226 to 788 times slower than a conventional core (section 7)
1	Cache of admitted blocks	Admit only fully verified blocks	About 3 times slower; hit rate and Checker cost decide viability (section 7)
2	Custom ops: pre-staged table, promotion by use	Validate configurations from a vetted set	Promotion narrowly beat pre-staging; rewiring per burst about 400 times slower (section 7)

Stage	Adds	What the Checker must do	Simulated evidence
3	Speculation, pipelined checking	Unchanged	Little gain unless synthesis plus check takes about one burst time or less (section 7)
4	Larger blocks	Symbolic equivalence, cost grows with size	Best cached near 16 instructions, reuse falls beyond (below)
5	Loop blocks	Loop invariant, termination bound, latency budget	1.13 to 1.48 times, against 3.3 times for windows of 16 (below)
6	Hierarchies and shared block libraries	Re-check on receipt	Not simulated

General and domain-specific. The same substrate would settle at different stages depending on the device. These are design expectations, not measured results. The simulated workload is a set of repeating loops, which is closest to the switch case.

- **Sensor or control gadget.** Likely stays at stage 0 or 1: a few repeating blocks, a small cache, a very small Checker, and hard latency met by the fast path. A fixed engine and a fixed invariant set narrow what the Checker must prove.
- **Network switch or similar appliance.** Likely climbs to stages 1 to 5, since traffic is dominated by a few repeating patterns and the hit rate is high.
- **Phone or general-purpose device.** Needs most stages and a rich Checker, because its workload is varied and a cache keyed on a wide State Tensor will hit less.

The two explorations that follow, larger blocks and loops inside blocks, are the stage 4 and stage 5 optimisation areas. They are simulated as a guide to where the tradeoffs lie, not as part of the core design.

Larger blocks. Nothing in the architecture limits a block to 16 instructions, so the simulator sweeps sizes up to 256, with synthesiser errors now applied per instruction (0.1%), so longer blocks fail more often.

Block size	Uncached	Uncached, superlinear check	Uncached, one-pass synthesis	Cached (1024 entries)	Cache hit	Interrupt worst case (ns)
4	790 times	778 times	651 times	3.3 times	100%	21
16	310 times	309 times	184 times	3.1 times	99%	33
32	231 times	239 times	107 times	4.6 times	98%	49
64	193 times	216 times	69 times	8.6 times	94%	78

Block size	Uncached	Uncached, superlinear check	Uncached, one-pass synthesis	Cached (1024 entries)	Cache hit	Interrupt worst case (ns)
128	178 times	222 times	50 times	15.4 times	86%	130
256	175 times	249 times	42 times	30.3 times	74%	271

Slowdown is against a conventional core. "Superlinear check" grows checker cost as the 1.5 power of block size, anchored at 16. "One-pass synthesis" costs a flat 2 µs per block whatever its length. The interrupt column uses an aborting fast path with the cache on.

Five lines of enquiry follow.

- **Fixed windows stop paying.** Hit rate falls from 99% at 16 instructions to 74% at 256, and slowdown rises from 3 to 30 times. Capacity is not the cause, since a separate run with an effectively unlimited cache gave the same hit rates. Fixed-size windows over a repetitive stream reuse poorly because of phase and ragged ends. This points to structure-aware blocks, such as bounded loops cached once per loop, which the Checker would have to verify with a loop invariant and not by straight-line shadow execution. This is simulated next.
- **Cached blocks need proof, not a test.** At 64 instructions, 5 wrong blocks were replayed from the cache even with full checker coverage. The simulated Checker tests a block on one state, and a block can pass on one state and fail on another. A cache admission check must hold for all states, which means symbolic equivalence, the proof-checker idea, and that is the expensive part.
- **The uncached floor is set by per-instruction costs.** Uncached slowdown flattens near 175 times because synthesis costs 100 ns per instruction. With one-pass synthesis it falls to 42 times, and what remains is mostly the Checker at 20 ns per instruction. A superlinear Checker makes large uncached blocks worse (175 to 249 times). Whether the engine emits a block in one pass or token by token therefore matters more than block size.
- **Interrupt latency grows with the burst.** Worst case goes from 21 ns at 4 instructions to 271 ns at 256. Large bursts probably need to be transactional, committing at the end and discarding on an urgent interrupt, or checkpointed within the burst.
- **Errors and blast radius grow with size.** At 0.1% per instruction, a 64-instruction block fails 6% of the time and a 256-instruction block 23%. Rejection catches most failures, but an escape does more damage.

Loops inside blocks. A loop is a block with a trip count, and it is admissible if the Checker can show it correct and terminating for any count. The simulator models a loop block as one loop body plus a trip count taken from the State Tensor. It is checked once, cached once per loop, and checkpointed every k iterations, which is where interrupts are serviced.

Block form	Slowdown	Cache hit	Interrupt p99 (ns)	Interrupt worst case (ns)
Fixed window of 16	3.33 times	99.2%	30	33
Fixed window of 64	10.49 times	93.5%	70	75
Loop block, checkpoint every iteration	1.48 times	98.8%	19	20
Loop block, every 4 iterations	1.21 times	98.8%	36	48
Loop block, every 16 iterations	1.15 times	98.8%	114	130
Loop block, every 64 iterations	1.13 times	98.8%	376	441
Loop block, whole loop	1.13 times	98.8%	410	715

- **Loop blocks cost far less than windows.** The cached unit is the loop, not an alignment-dependent window, so the hit rate stays at 98.8% and slowdown falls to 1.13 to 1.48 times. The workload is built entirely from counted loops, so this is a best case.
- **The checkpoint interval is the interrupt sensitivity dial.** Worst-case latency is roughly the checkpoint interval times the body length: 20 ns when checkpointing every iteration, 130 ns every 16, 715 ns for the whole loop. The Checker could set the interval from a latency budget stated in the invariants, so the latency bound becomes something it enforces, not a property of the loop.
- **Random testing is not a proof.** With 2% error per instruction, a Checker that tested cached 64-instruction windows on one state let 13 wrong blocks run from the cache. Requiring agreement on 8 further random states cut that to 4 but not to zero. A real symbolic proof, including a loop invariant and a termination bound, is what the architecture needs. The loop blocks showed no escapes in either case, but with short bodies and a small sample that says little.
- **The proof has to pay for itself.** A proof costing 50 times the cheap check leaves loop blocks at 2.2 times slowdown, 500 times gives 13 times, and 5000 times gives 64 times, on a workload of only 0.65 ms of conventional execution. A long proof also competes with interrupts. Each abort restarts it, so after 8 aborts it is made non-preemptible, which then delays interrupts for the length of the proof (worst case 8 µs at 50 times, 176 µs at 500 times).

- **Interrupts of interrupts.** At one interrupt per 2 µs, 3 to 8% of arrivals found a handler already running. At one per 200 ns it was 31 to 60%, though part of that is backlog queued behind a long proof. Nested preemption is not modelled. A priority scheme with bounded nesting depth, with handlers themselves being bounded blocks, would make the worst case the sum of the levels' bounds.

9. No Application, No Operating System

In a conventional device the application is the stored program of intent, and the operating system is the stored program of resource policy: scheduling, isolation, drivers, files. UMA has neither as a stored artifact. What remains is a substrate that is the same on a network switch, a phone or a small gadget. Devices differ in three things only: their intent, their peripherals, and their invariants.

Conventional role	UMA counterpart
Application	Intent held in the State Tensor, refreshed from inputs. A switch's intent is to forward frames per learned topology and policy; a gadget's is to sample, filter and report
Scheduler	Demand clock and synthesiser, which emit whatever the moment needs
Device drivers	Synthesised peripheral-handling blocks, confined to ranges the Checker approves
Memory protection and isolation	Checker rules on what any block may read, write or reconfigure
Logging, accounting, files	The trace log, with inputs and time stamps
Install, update	A change of intent, invariants or engine parameters, not a new binary

What does not disappear. Persistent policy has to live somewhere, and in UMA it lives in the Checker's invariants and in whatever shapes the engine. These are the residual operating system. The other fixed elements are the interrupt fast path and its hardwired recovery blocks, the boot ROM that sets the initial intent and invariants, the hardware timers and interrupt sources, and the Trace Tap with its time reference. The engine's parameters also carry device knowledge, such as how to drive a particular display. The design aim is to keep this fixed set small and verifiable and everything else ephemeral.

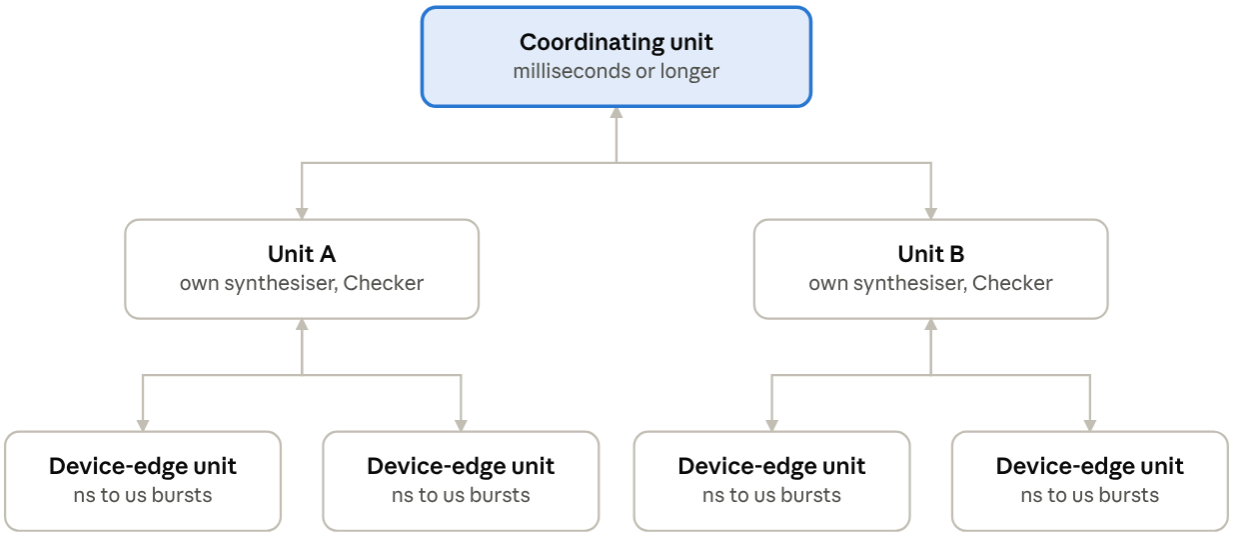
Authorship moves, it does not vanish. Intent and invariants are still specifications, and someone or something must write them. UMA moves authorship from instructions to goals and constraints. How those are expressed, and how they are updated on a deployed device, is the central open design question.

Boot and trust. Something must establish the initial intent and invariants and protect them from tampering. That root of trust is a small fixed element, which is acceptable as long as the paper says so.

10. Coordinated Hierarchies

UMA units compose. One unit's intent can be supplied by another unit, so a device can be a hierarchy of units, each with its own synthesiser and Checker. Intent flows down and state and trace logs flow up. Each level runs at its own timescale: device-edge units burst in nanoseconds to microseconds, coordinating units deliberate over milliseconds or longer.

Intent flows down, state flows up, and every unit checks



Arrows run both ways: intent down, checked on receipt; state and traces up.

hierarchy · 7 units, 3 levels

Each unit accepts intent from above only after its own Checker approves it, and reports state and trace logs back up.

Bounded delegation. A unit checks incoming intent against its own invariants before accepting it. A wrong or compromised parent therefore cannot push a child outside the child's invariant set. Safety is local and composes, even though correctness of the overall behaviour does not.

Coordination by goals and state, not code. Peers and parents exchange intent and state summaries, never instruction streams. There is nothing to deploy: a unit is re-purposed by changing what it is asked to do. Incoming messages are data, validated on receipt like any other input.

Fast and slow paths. Reflexes stay local, as with the interrupt fast path. Slower, more expensive synthesis happens higher up, where its latency is tolerable.

What this does not solve. Hierarchies bring distributed-systems problems: consistency, delay, feedback stability, and untrusted peers. Checkers at each level bound the damage but do not guarantee liveness or good coordination. How intent summaries compress as they pass upward, and how mixed hierarchies containing conventional nodes behave, are open.

11. Security and Safety Model

Removing the stored binary removes one class of attack, tampering with a file at rest, and changes where the others sit.

- **Input as attack surface.** Peripheral and network data shape the State Tensor, so crafted inputs can steer synthesis in ways analogous to prompt injection. Inputs need validation and the Checker must not trust the engine's intent.
- **Denial of service by vacuous blocks.** An attacker could trap the engine in an endless cycle of valid but computationally empty blocks. The Checker therefore also enforces structural resource limits and bounds execution depth, which acts as a progress watchdog.
- **Auditability by trace log.** The recorded block stream and input log make behaviour inspectable after the fact. This restores the audit trail that the absence of a static binary would otherwise remove. The log is hash-chained and signed by a hardware key so that it is tamper-evident, and its time stamps come from a hardware-isolated reference (Appendix B), typically a local oscillator that can free-run on its own.
- **Bounded blast radius.** A 4 to 16 instruction block can still perform a stray store or a bad GPIO write, so bounds, permitted opcodes and write-region limits are enforced by the Checker, not left to the engine.
- **Cache poisoning.** An exploit that satisfies a loose Checker rule will be replayed on every cache hit, as the simulation shows. The main defence is admission: only blocks that passed the full check enter the cache. A second defence is periodic re-audit of cached blocks against the trace log, using a stronger, slower check than the one that admitted them, with eviction on any anomaly or invalid transition. Re-audit limits recurrence, not the first occurrence.

- **Model protection.** Whatever holds the engine's behaviour is a high-value target and needs the same integrity and extraction protections as firmware.

12. Related Work, Open Questions and Conclusion

Related work. UMA sits near several established areas: JIT and tracing compilers, dynamic binary translation and code-morphing processors, record-and-replay systems, coarse-grained reconfigurable arrays and partial FPGA reconfiguration, self-timed and clockless logic, and program synthesis. For hibernation, relevant work includes non-volatile and normally-off processors and intermittent computing. For the no-operating-system view, relevant work includes unikernels, library operating systems, exokernels and capability hardware such as CHERI. For hierarchies, it includes layered control architectures such as subsumption and multi-agent control hierarchies. Its distinguishing element is a learned synthesiser as the instruction source, checked at every burst. A fuller treatment should position UMA against each.

Open questions.

- How is intent expressed and encoded in the State Tensor, and what language do the Checker's invariants use? This is the specification the Checker verifies against.
- Which real workloads have block working sets that fit a cache, and how well does a cache keyed on a quantised State Tensor hit compared with the perfect keying used in the simulation? Which stage of the roadmap in section 8 does each class of device settle at? The simulation shows hit rate and checker cost per block are decisive.
- What block size and structure is right (straight-line, bounded loop or transactional), and what does the Checker prove for each? Can it enforce an interrupt latency budget by choosing a loop block's checkpoint interval? The simulation shows larger bursts cost more once cached, but that was with an oracle synthesiser.
- How is the engine trained, and how do trace-log and feedback loops update it without breaking reproducibility? Does it emit a block in one pass or token by token? That sets the uncached cost floor.
- How is cache admission restricted to fully verified blocks, and what stronger check catches a cached block that the admitting Checker missed?
- What latency does the interrupt fast path reach in practice, and can a running burst be made safely abortable? How should nested interrupts and priorities work when handlers are themselves blocks?
- In hierarchies, how do intent summaries compress as they pass upward, how stable are feedback loops across levels, and how do mixed hierarchies with conventional nodes behave?

- What root of trust fixes the initial intent and invariants, and how are they updated on a deployed device? If a device hibernates, how is the dormant state's integrity checked on resume?

Conclusion. UMA is a proposal for a machine whose instructions are formed at the moment of need, checked, executed and recorded, not authored and stored. It does not abolish software so much as move its content into the synthesiser, its record into the trace log, and its authorship from instructions to intent and invariants. The same substrate can be a switch, a phone or any gadget, and units can be composed into hierarchies that coordinate by goals and state, not code. A first simulator (section 7) shows that cache hit rate and checker cost per block decide viability. The next steps are to specify the Checker's invariant language, including loop invariants and latency budgets, replace the oracle synthesiser with a real model, and simulate a small hierarchy of units.

Appendix A. Worked Example: Boot to Hello World

The example is deliberately small. It assumes the display is already configured, so only the text path is traced. Every boot is cold: the block cache is empty, and "never again" below means never again within this run.

Pass 1: the minimum system. Power is applied and the boot ROM loads the Checker invariants and the initial intent ("write the string to the display"). The Synthesis Engine emits short straight-line blocks of a few instructions, one per character or small group. The Checker applies bounds and opcode checks and runs a test against the current State Tensor, then the block runs as a micro-burst and the State Tensor is checkpointed. Nothing is cached yet, so every pulse pays for synthesis and checking, about 3 µs per pulse against about 50 ns for native code in the simulator's placeholder timings, dozens of times slower. That is acceptable at this stage because the aim is a working, auditable system, not speed.

Pass 2: the optimised form. Once the system runs, the repeated pattern (fetch character, write, advance) is emitted as a single loop block with a trip count and a loop invariant. The Checker proves it once, the block cache admits it, and later iterations cost a lookup. The roughly 1.15x figure from the simulation applies only to this steady-state reuse with a cheap proof; it does not describe the first run.

Extension: a blinking cursor. A hardware timer fires every 500 ms. Through the interrupt fast path it sets a cursor-toggle bit in the intent, and the engine emits a four-instruction block: LW the cursor cell, XOR with a preloaded mask, SW it back, ACK the timer. The block is cached after its first verification. Real-time stamps in the trace log would show the timer drifting slightly from exactly 500 ms, which is the temporal track doing its job.

Phase	What happens	Cost profile
Power on	Boot ROM loads invariants and initial intent	Fixed, tiny
Pulse	Engine synthesises a block from the State Tensor	Dominant when uncached
Check	Bounds, opcodes, depth, equivalence (test or proof)	Grows with block size and proof strength
Burst	Block runs at full hardware speed	Negligible
Checkpoint	State Tensor sampled, log entry written	Small
Idle	Demand clock inhibited until the next timer interrupt	Zero

Appendix B. Spatial-Temporal Binding: The Dual-Track Trace Log

The trace log records what ran in two tracks that are bound together per burst.

Track	Content	Properties
Spatial	Block IDs, opcodes, registers and memory touched	Contains no time, so identical blocks give identical records and remain cacheable
Temporal	Real-time stamps at each demand-clock edge, plus stamped external inputs	Taken by the Trace Tap from a hardware-isolated source

The time reference is local and isolated. The stamps come from a local oscillator, such as an atomic clock, that can free-run indefinitely. External sources such as GPS or PTP are optional disciplining inputs for aligning nodes, and losing or distrusting them leaves the local timeline intact, with some drift. The reference is kept structurally isolated from the engine so the engine cannot rewrite the record.

Accuracy is bounded by the method. Cross-node alignment is as good as the synchronisation allows, typically tens of nanoseconds to microseconds, and no better. The paper makes no claim of exact simultaneity.

Benefits.

- Cross-machine coherence: logs from several UMAs in a hierarchy can be merged on one timeline, within the stated bound.
- Replay with the world in it: the block sequence plus stamped inputs reproduces a run including its timing context.
- Runaway detection after the fact: bursts that overran expectations are visible in the stamps.
- Explicit gaps: time spent synthesising, idle or unpowered appears as a gap rather than being hidden.

Design points. Log one entry per burst with block ID, start and end stamps and inputs. Hash-chain the entries and sign them with a hardware key so the log is tamper-evident. Keep the Trace Tap and time reference in hardware, as part of the fixed residual policy.